Formal Language Automata theory AD 23412

Unit-3

Context Free grammar and Push Down Automata

Topics :

- Types of Grammar ✓
- Chomsky's hierarchy of languages ✓
- Context Free Grammar (CFG and languages) ✓
- Derivations and parse trees ✓
- Ambiguity in grammar and language ✓
- Push Down Automata (PDA) : Definition ✓
- Moves ✓
- Instantaneous descriptions ✓
- Languages of Push Down Automata ✓
- Equivalance of Pushdown automata and CFG - CFG to PDA - PDA to CFG ✓
- Deterministic Pushdown Automata ✓

Types of Grammar

- Production Rule
  - → Type 0 - Type 3
- No of derivations
  - → Ambiguous
  - → Unambiguous
- No of strings
  - → Recursive
  - → Non Recursive

Chomsky's Hierarch of Language

Type 0 -> Regular grammar
Type 1 -> Context Free grammar
Type 2 -> Context Sensitivity Grammar

Context free grammars

*) Context free languages are generated by some context free grammer

*) The set of all CFL is identified to the set of languages accepted by Push Down Automata

It has 4 Tuples of CFG

$G = \{V, \Sigma, S, P\}$

V -> set of Non terminal Symbols
Σ -> Set of terminal Symbols
S => Start Symbol
P -> Production ~~rule~~ Rule

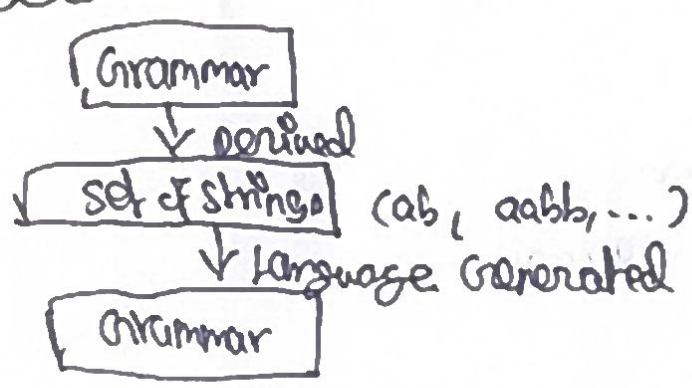
Ex: CFG for no of a's and b's are equal

$G = \{(S,A), (a,b), (S \to aAb, A \to aAb \mid \epsilon\}$

we shall expand the rule to find the language

$S \to aAb$
$\to aaAbb$ (by $A \to aAb$)
$\to aaaAbbb$ (by $A \to aAb$)
$\to aaabbb$ (by $A \to \epsilon$)
$\to a^3b^3 \Rightarrow \{a^n b^n\}$

Derivation parse tree



~~So far we saw derivation tree, now it time for parse tr~~

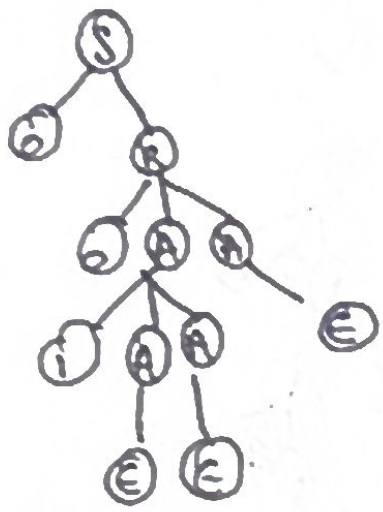
So far we have seen parse tree now lets see derivation tree which is also parse tree.

A derivation tree or parse tree is an ordered rooted tree that graphically represents the semantic information of strings derived from a context free grammar

Problem 2

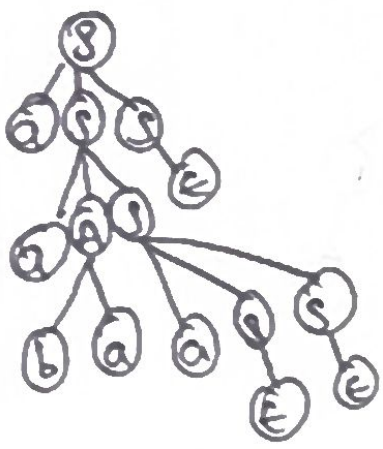
$G = \{V, T, P, S\}$ where $S \to 0B$, $A \to 1AA$, $B \to 0AA$

Start = root

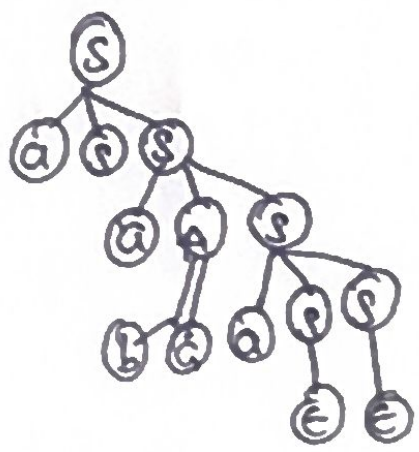


3 $S \to aAS \mid aSS \mid \epsilon$ $A \to SbA \mid ba$

Left most tree



right most tree



aabaa

Problems with derivation & parse

(1) The grammar $E \to E+E \mid id$ is ambiguous to prove $w = id + id + id$

Left Most Derivation

$E \to E+E$

$E \to E+E+E$ ($\because E \to E+E$)

$E \to id+E+E$ ($E \to id$)

$E \to id+id+E$ ($E \to id$)

$E \to id+id+id$ ($E \to id$)

Right Most Derivation

$E \to E+E$

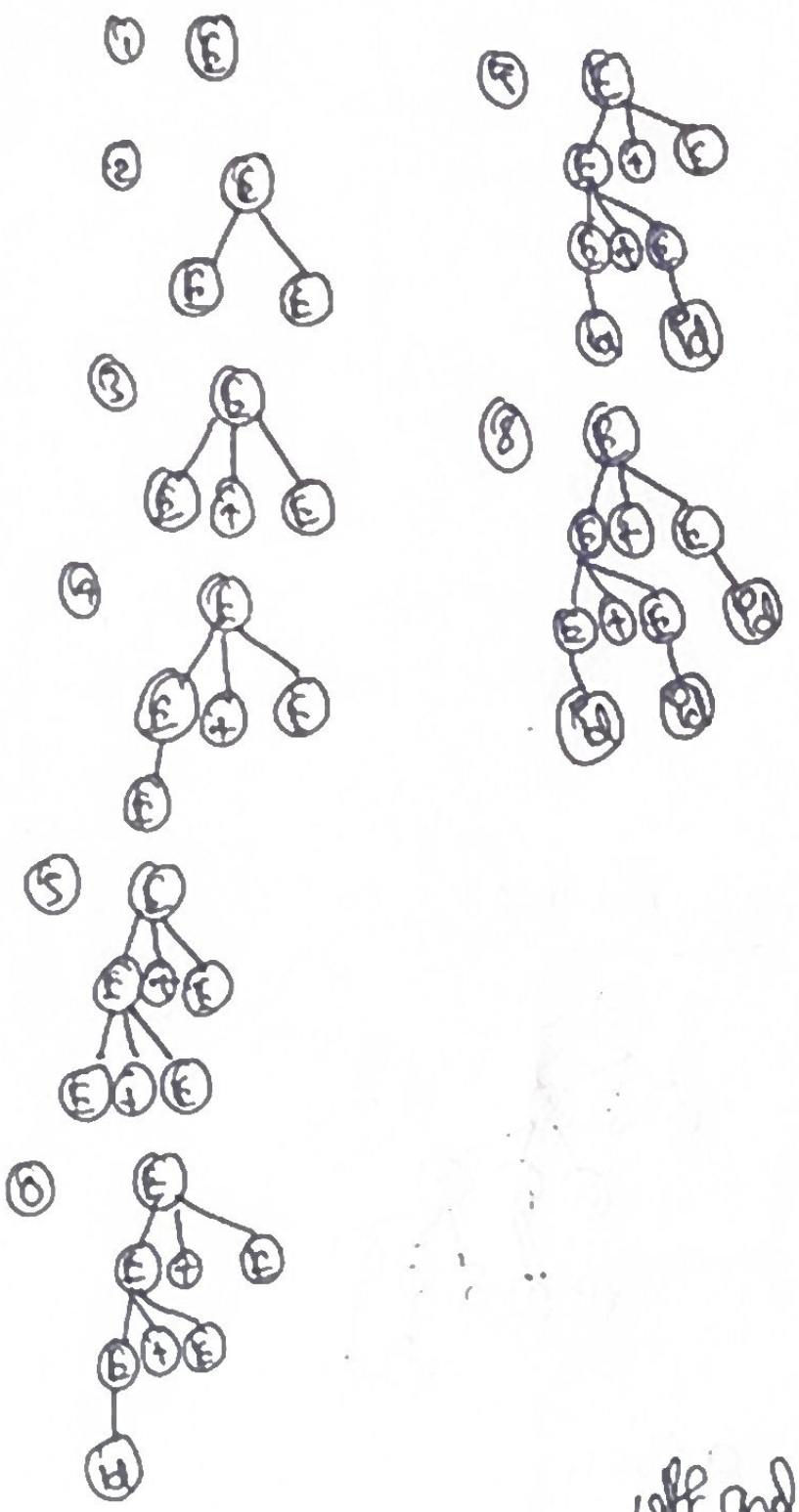
$E \to E+E+E$

$E \to E+E+id$

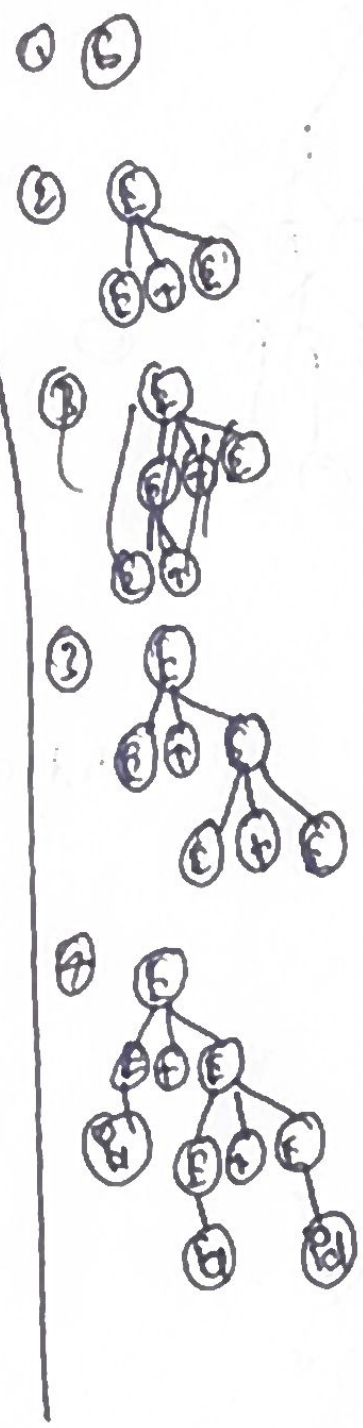
$E \to E+id+id$

$E \to id+id+id$

Left Most parse tree



Right Most parse Tree



Left and right tree that have equal number of steps but for simplicity the steps are less

Ambiguous Grammar

A Grammar is said to be ambiguous if there exists two or more derivation trees for a string w (defn applies to ambiguous language too)

Same as parse and derivation tree problems, we would just be asked if they are ambiguous.

Problem 3

construct left and Right Most trees for string 1100101

Grammar

S -> 0A | 1B

A -> 1 | 1S | 0AA

B -> 0 | 0S | 1BB

Derivation (left most)

S => 1B

S -> 1(1BB) (B -> 1BB)

S -> 11(0)B

S -> 110(0S)

S -> 1100(1B)

S -> 11001(0S)

S -> 110010(1B)

S -> 1100101(0)

String: 1100101

Right most

S -> 1B

S -> 1(1BB)

S -> 1(1B0)

S -> 11B0S

S -> 11B0(1B))

S -> 11(0S)(010)

S -> 11(0(1B)(010)

S -> 1101(0)010

S -> 1100101

Now than generate parse tree according to the derivation

Push Down Automata (PDA)

A pushdown automata (PDA) is a way to implement a context free grammar, in a similar way as we design finite automata for regular grammar

$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

where

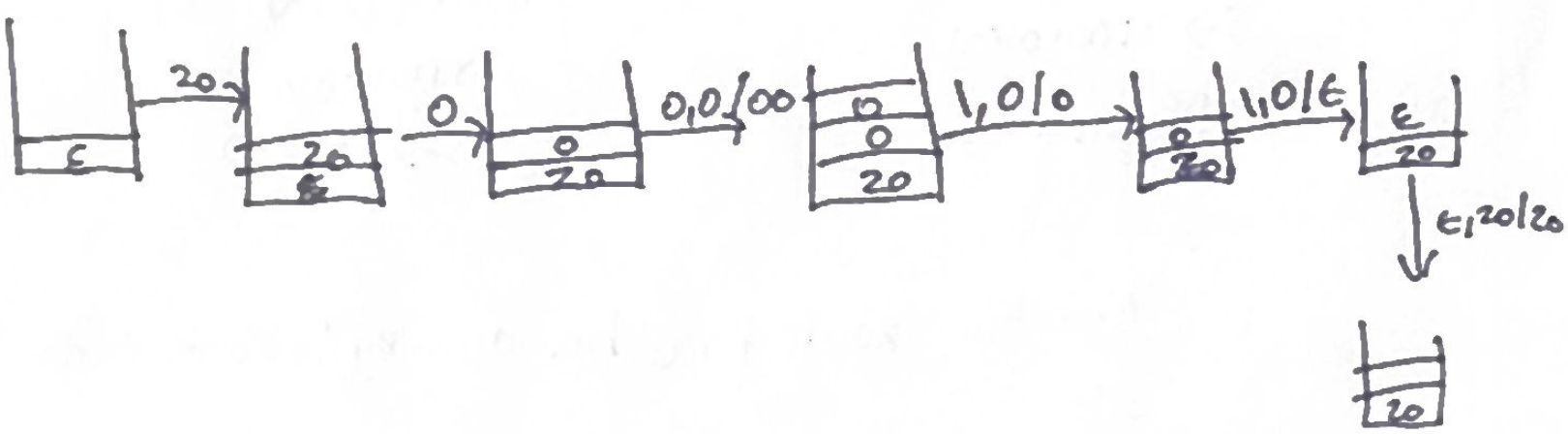
Q = A finite set of states
Σ = A finite set of input symbols
Γ = A finite stack alphabet
δ = Transition function
$q_0 \rightarrow$ initial state
$F \rightarrow$ final state
$Z_0 \rightarrow$ stack symbol

Problems

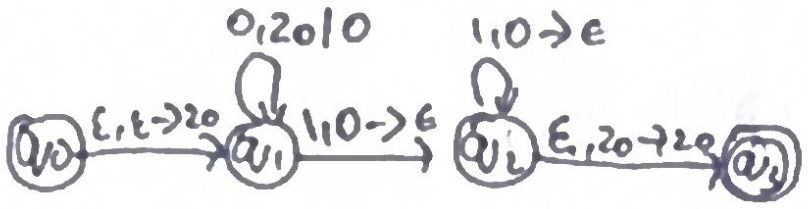
Construct PDA, $L = \{0^n 1^n \mid n \geq 1\}$

$L = \{0011, 01 \ldots\}$

Step 1: Draw stack diagram:



Step 2 : Draw automata



Step 3 : Instance design for (0011)

$(q_0, \epsilon, Z_0) = (q_0, \epsilon Z_0)$

$(q_1, 0, Z_0) = (q_1, 0Z_0)$

$(q_1, 0, 0) = (q_1, 00)$

$(q_1, 1, 0) = (q_2, 0, \epsilon)$

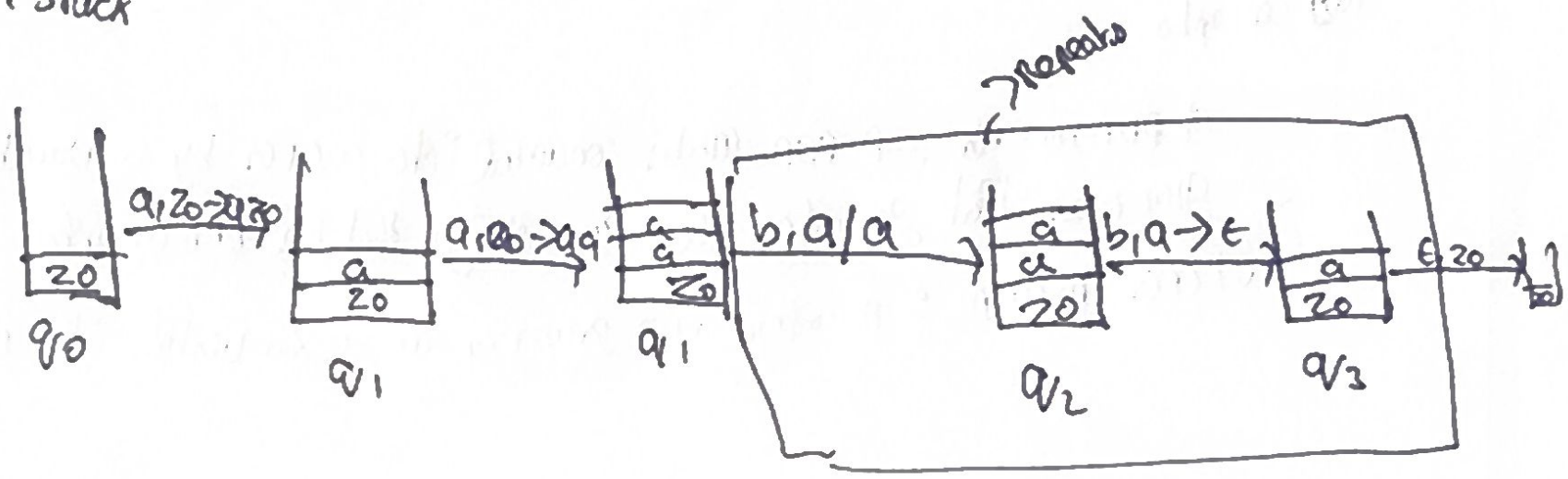
$(q_2, \epsilon, Z_0) = (q_3, Z_0)$ → final state is reached

Then this are valid

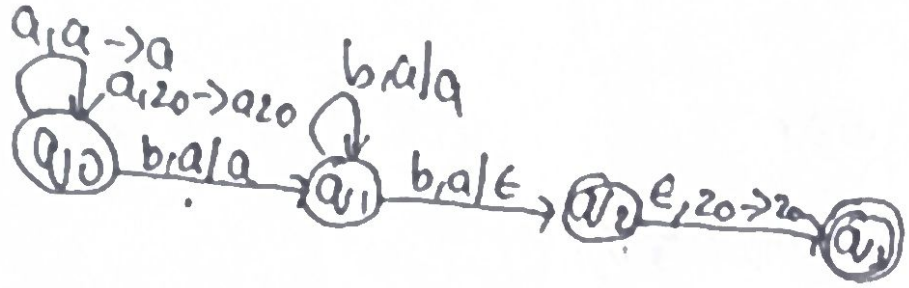
② $L = \{a^n b^{2n} / n \geq 1\}$

$L = \{abb, aabbbb \ldots\}$

Step 1 : Stack

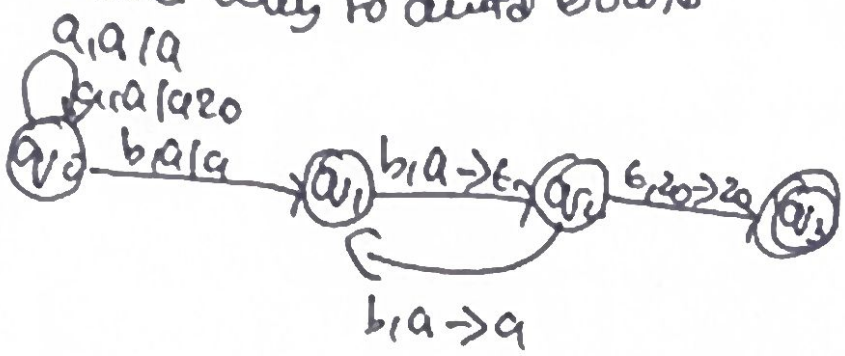


Step 2: Automata



Step 3: Same as before

Alternative way to avoid errors



— — —

Equivalence of PushDown automata and context free grammar

PushDown Automata and context free Grammars (CFGs) are equivalent because both define the same class of languages: context-Free Languages (CFL s) This means any language generated by a CFG can be recognized by a PDA and any language accepted by PDA can be generated to CFG

Key Concepts:

1) PDA to CFG : A PDA can be converted into a CFG by constructing a grammar that generates the same strings that the PDA accepts

2) CFG to PDA : A context free grammar can be converted into PDA

PDA to CFG conversion

Steps:

1) Simplify the PDA
2) Build the CFG

Main Idea:

Consider two states p and q in the PDA
Could we go from p to q without stack underflow and maintaining an empty stack at the beginning and end? or if something were already on the stack they would never be touched

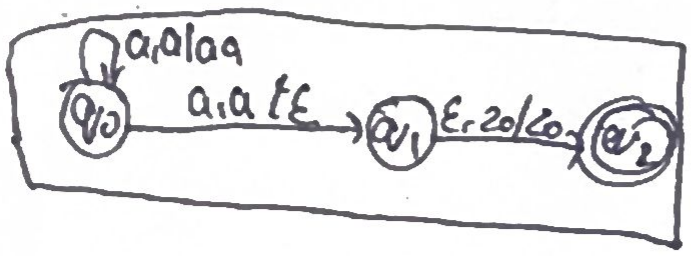
What strings would do that?

We will create a nonterminal A_{pq} in our grammar. A_{pq} will generate exactly those strings that will take us from p to q maintaining all the above stack conditions

— — topic passed for aficionados — —

Deterministic Push Down Automata

A PDA is said to be deterministic if all derivations in the design has to give only single move



Non deterministic

A Deterministic PDA is a tuple $M: (\Sigma, \Gamma, Q, \delta, q)$

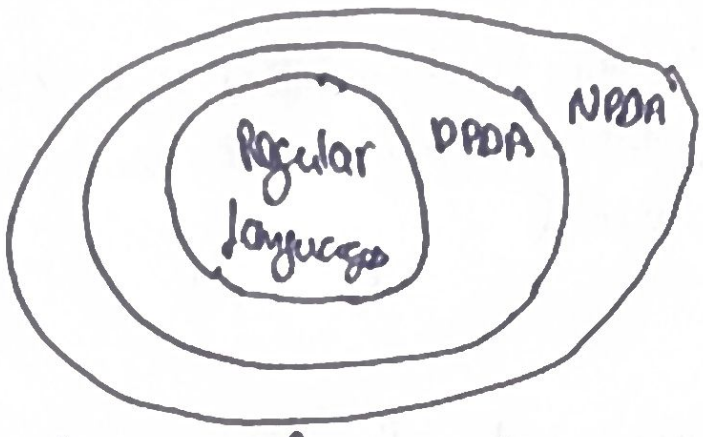
$\Sigma \rightarrow$ It is a finite set which does not contain any blank symbol

$\Gamma \rightarrow$ a finite set of stack alphabet

$Q \rightarrow$ Set of states

$q \rightarrow$ start state

$\delta \rightarrow$ a transition function denoted as $- \delta : Q \times$



Same Defn can be applied DCFG/L, NDCFG/L

Non-Deterministic PDA

NPDA is powerful than DPDA

*NPDA has 7 tuples

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$